CSCI 2110 Data Structures and Algorithms

Module 9: Hashing and Hash Tables

DALHOUSIE UNIVERSITY

MOTIVATION

- Name the three most important and common operations that you would perform on any data structure.

- **SEARCH, SEARCH AND SEARCH!**
- How did we do in terms of the time complexity of search on different data structures?
  - Unordered List: **O(n)**
  - Ordered List: **O($\log_2$n)** (because of binary search)
  - Binary Tree: **O(n)** (because it can be just a linear set of nodes)
  - Binary Search Tree: **O($\log_2$n)** if the tree is complete (balanced) – Good tree!
    O(n) if the tree is not balanced – Bad tree!
  - Heaps: **O(n)** key can be anywhere on the tree

- **CAN WE DO BETTER???**

Can we aim for O(1) complexity for search?

- Let's set ourselves a tall order: O(1) for search (at least on the average).
- We know that in an array, given its index, we can retrieve an element in O(1) time.
- Can we look back to the array for help?
- How can we put a set of keys onto an array so that the search complexity is O(1)?

Can we aim for O(1) complexity for search?

- Suppose we have an arbitrary set of keys 2, 8, 7, 4, 0.
- Remember that each key is associated with an object: if you get the key, you get the object.
- If we store the keys in the array arbitrarily, we need O(n) time → Sequential search

2	**8**	**7**	**4**	**0**					
0	1	2	3	4	5	6	7	8.	9

- If we store the keys in the array sorted, we still need O($\log_2$ n) time → Binary search

0	**2**	**4**	**7**	**8**					
0	1	2	3	4	5	6	7	8.	9

- **BUT WHAT IF WE <u>MAKE THE KEY THE INDEX</u>?**
- **Time for search: O(1) (insert exclamations here).**

Here's the idea:

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

Keys:
2, 8, 7, 4,0

0	
1	null
2	
3	null
4	
5	null
6	null
7	
8	
9	null

What is the catch?

- In our example, we chose the keys to be 2, 8, 7, 4, 0 and hence we declared an array of size 10.
- **What if the keys are 1, 43, 102, 131,15143 and 19992?**
- Would you declare an array of size 20000?
- It appears that whatever you gained in time complexity was lost in space complexity.
- Can we have the best of both – good time complexity and good space complexity?
- ***The trick: <u>MAP the key to an index!</u>***
- **The process: HASHING**
- **The data structure: HASH TABLE**

Hashing and Hash Table - Definition

- A hash table is a storage array.
- A hash function maps a key to a specific index on the hash table.
- Hashing is the process of storing objects in the hash table using a hash function.

A simple hash function or mapping function can be written as:

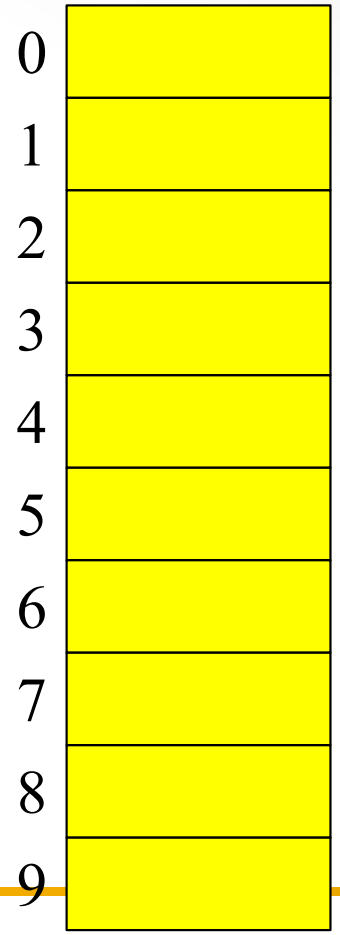
key % N

where N is the table size.

For example, if the table size is 10, then we get our index as follows:

index = key % 10

Example



Suppose we have these 5 keys:
9876, 234, 5672, 89990, 6777

We choose a table of size 10 (enough to fit the number of keys).

Then we apply the hash function

$$index = key \% 10$$

to map the keys to the table.

Example

Keys:
9876, 234, 5672, 89990, 6777

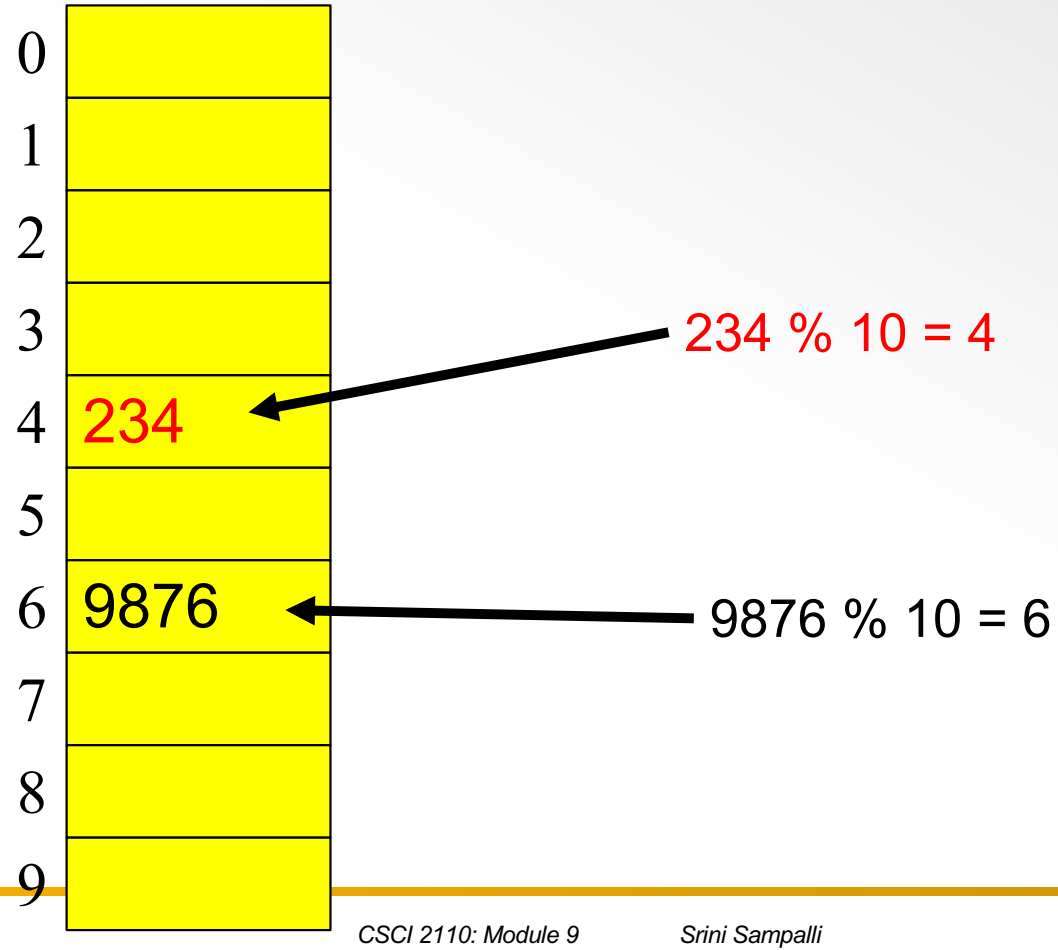
9876 % 10 = 6

Key 9876 goes to location 6

0	
1	
2	
3	
4	
5	
6	9876
7	
8	
9	

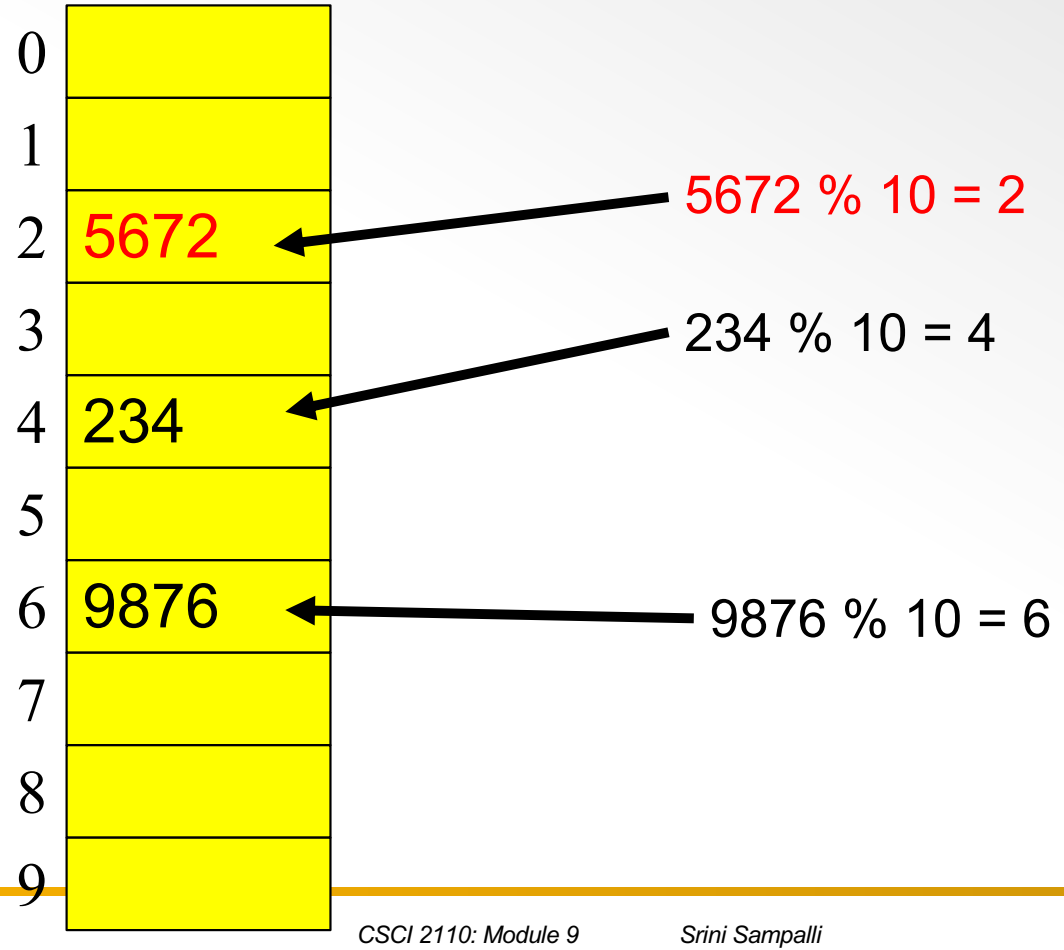
Example

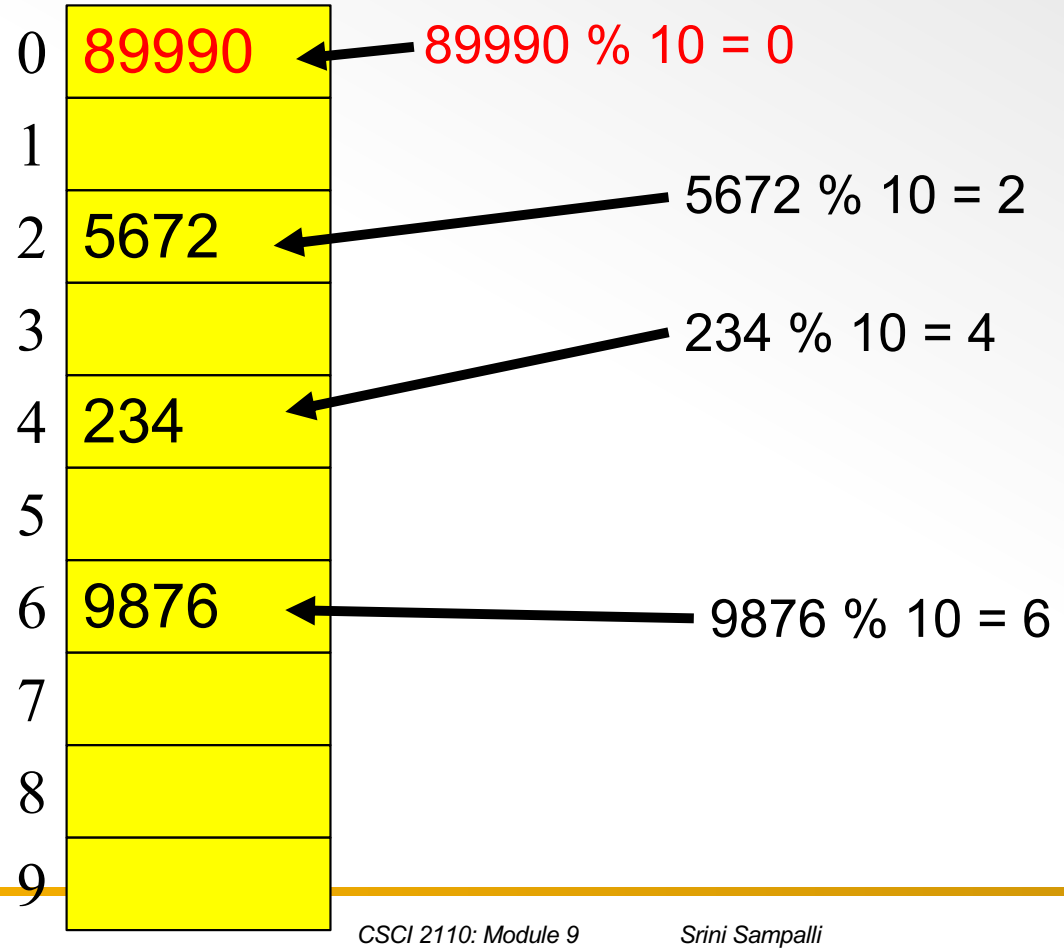
Keys:
9876, 234, 5672, 89990, 6777



Example

Keys:
9876, 234, 5672, 89990, 6777



Example

Keys:
9876, 234, 5672, 89990, 6777